EE992 (Neural Networks and Deep Learning) Semester 2 Deep Learning Assignment

Andrew Law and Pittayut Benjamusutin

***Abstract*—This report explores the exploration, development and evaluation of deep learning models for classifying images within the CIFAR-100 dataset. The CIFAR-100 dataset contains 1000 classes of 600 images each, with accompanying labels.**

The main objective is to compare the performance of ResNet50V2 model with different hyperparameters, as well as several models from scratch, whilst investigating the effects of transfer learning, fine-tuning, and regularisation.

Experimental results shows that ResNet50V2 performs the best with a 67.04% accuracy, when dropout is set to 0.6, weight decay to 0.3, L2 regularisation to 0.9, initial training learning rate to 1e-3 and fine-tuning rate to 1e-5. The study highlights that finding a balance of hyperparameters leads to optimal results, preventing overfitting whilst mitigating over-regularisation.

The study concludes that a pre-trained ResNet50V2 is suitable at classifying the CIFAR-100 dataset. For the future, models such as vision transformers and more hyperparameter analysis could improve results.

I. Introduction

IMAGE CLASSIFICATION is a fundamental computer vision task that enables computational systems to identify and categorize visual content within digital images. This process typically employs machine learning models to assign predefined labels or categories based on extracted visual features. While traditional techniques such as support vector machines (SVMs) and handcrafted feature extraction have historically addressed this task, modern approaches increasingly rely on deep learning (DL) models. Convolutional Neural Networks (CNNs) have dominated ever since AlexNet's introduction around 2012 in ILSVRC. It achieved breakthrough performance on ImageNet's 1.2 million high-resolution images with 60 million parameters and 650000 neurons [x1]. This was the catalyst to widespread adoption of DL in image analysis.

The CIFAR-100 dataset, a widely used benchmark in image classification research, consists of 60,000 32×32 RGB images evenly distributed across 100 fine-grained classes [2]. This project focuses on developing a deep learning framework to classify CIFAR-100 images into their respective fine-grained categories. The primary objectives are threefold: (1) Achieving a minimum classification accuracy of 60%, reflects pragmatic balance between performance and computational feasibility; (2) Ensuring robust generalization to unseen data, prioritizing testing accuracy over adversarial robustness.

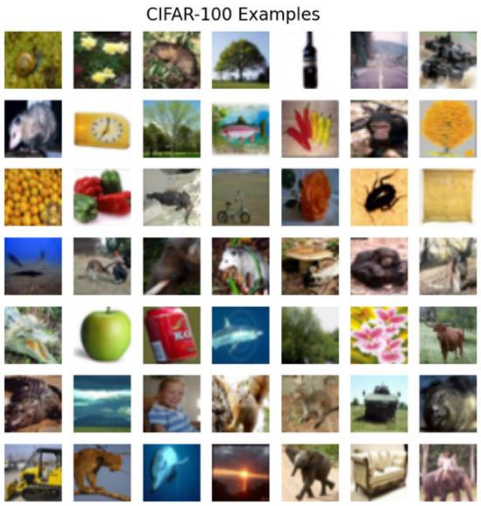

Figure 1 - CIFAR-100 Image Examples

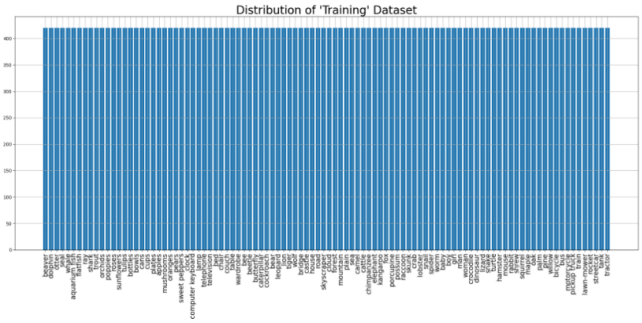

Figure 2 - Training Dataset Classes Distribution

To meet these goals, we experiment with different neural network architectures, optimization strategies, and training hyperparameters. The results are evaluated using metrics like testing accuracy and f1-score and visualized through loss and accuracy plots, as well as confusion matrices for class-wise performance and identify misclassifications. By iterating through these experiments, we aim to identify a streamlined approach that balances performance with practicality, offering insights into effective model design for small-scale image classification tasks.

II. Literature Review \ Background

A. Convolution Network

CNNs are a class of deep learning models optimized for image processing and pattern recognition. They consist of multiple layers, including convolutional layers that apply small filters (typically 3×3 or 5×5) to extract spatial features such as edges and textures, generating activation maps that highlight detected patterns in different regions. Pooling layers (e.g., max pooling) reduce spatial dimensions while preserving critical features, improving computational efficiency. Non-linear activation functions like ReLU and Softmax introduce complexity, enabling the network to learn intricate representations, where ReLU helps prevent vanishing gradients and Softmax takes raw outputs and ensures all sum to one for output classification layer. Finally, fully connected (FC or Dense) layers aggregate extracted features for classification, allowing CNNs to achieve high accuracy in tasks such as image recognition and object detection [1], [3].

B. Learning Process

The learning process involves forward and backward passes. Initially, weights are randomly initialised to promote unbiased learning. During the forward pass, input training data propagates through the network, generating predictions. The difference between predictions and ground truth is computed by a loss function (e.g., cross-entropy for multi-class classification due to one-hot encoding). To minimize this loss, backpropagation applies the chain rule to compute gradients with respect to network parameters (weights and biases). Gradient descent then adjusts these parameters in the direction of the negative gradient, iteratively reducing the loss [4].

The loss landscape can be visualised as a surface with multiple minima. While the goal is to reach the global minimum, optimization may stall in local minima. Techniques such as momentum and adaptive learning rates help mitigate this issue. Additionally, regularization terms (e.g., L2 regularization) may be incorporated into the cost function to prevent overfitting by penalizing large weights.

C. Training Optimization

Modern optimization algorithms automate parameter tuning, though hyperparameters (e.g., learning rate) often require empirical tuning. Mini-batch gradient descent strikes a balance between stochastic gradient descent (SGD) and full-batch updates, offering stable convergence with efficient memory usage. Typical batch sizes range from 50 to 256 [4].

Momentum accelerates convergence by smoothing gradient updates, reducing oscillations in steep regions of the loss landscape [5]. RMSprop adapts learning rates per parameter by normalizing gradients using an exponential moving average, improving training stability. Recommended settings include a momentum of 0.9 and a learning rate of 1e-3 [4], [5].

Adam combines momentum and RMSprop, dynamically adjusting learning rates while maintaining memory efficiency. It includes bias correction to stabilize early training and requires minimal hyperparameter tuning compared to SGD [6].

D. Batch Normalization and Residual Network

Batch normalization normalizes layer inputs per mini-batch, mitigating internal covariate shift (changes in input distribution during training). This accelerates convergence and improves stability [8].

Very deep networks suffer from vanishing gradients, where backpropagated gradients diminish across layers. Residual Networks (ResNets) overcomes degradation issues with skip connections to improve gradient flow and enables effective very deep network training. By learning residual mappings instead of direct transformations, it optimises feature reuse, enhances feature extraction and improves generalisation to outperform models like VGG, whilst having fewer parameters [10]. ResNet50V2 was selected for enhanced training stability and slight performance gain on ImageNet. Unlike ResNet50 it applied batch normalisation and pre-activation before weight layers improving training efficiency [11].

E. Transfer Learning and Generalization

Transfer learning leverages pre-trained models (e.g., ResNet50V2 on ImageNet) to improve generalization on new datasets. Overfitting is countered via techniques such as data shuffling randomizing sample order to prevent bias. Early stopping halts training when validation loss plateaus [4]. These methods ensure robust model performance on unseen data while maintaining computational efficiency.

III. Methodology

A. Data exploration and preprocessing

1) Data Splitting and Preprocessing

Since the CIFAR-100 dataset provides an 80% training and 20% testing split, adjustments were made to align with the desired proportions. Dataset was partitioned into 70% training, 15% validation, and 15% testing subsets to facilitate model development, hyperparameter tuning, and unbiased performance evaluation.

Stratified sampling ensured proportional representation of all 100 classes in each subset, mitigating class imbalance and preventing model bias toward overrepresented categories. The final distributions were verified by plotting class frequencies, confirming uniform representation across splits, as shown in Figure 2.

2) Data Augmentation

Given the limited number of samples per class (600 images), data augmentation was applied to enhance generalization. The augmentation pipeline included: Random rotation (±15 degrees) to introduce viewpoint variations; Horizontal flipping (50% probability) to increase orientation diversity; Brightness adjustments (±20% variation) to simulate lighting changes.

These transformations were applied dynamically during training or fine-tuning to artificially expand the dataset while preserving label integrity.

3) Input Normalization

By dividing by 255, pixel values were normalised from their initial range of [0,255] to [0,1]. This scaling prevents ReLU-based networks from experiencing unsteady gradients due to overly large activations and

ensures constant input magnitudes across features, which speeds up convergence.

B. *Training Environment*

The training environment was carefully configured to optimize model performance while maintaining computational efficiency. For loss computation, both categorical cross-entropy and sparse categorical cross-entropy were evaluated to accommodate different label encoding formats. The optimization process employed with Adam optimizer. A comprehensive regularization strategy was implemented, combining L2 weight decay to constrain parameter magnitudes, dropout layers to prevent co-adaptation of features (λ=0.0001 and p=0.5 for custom CNN). Dynamic learning rate adjustment was achieved on custom CNN through ReduceLROnPlateau scheduling (reduction factor = 0.1, patience = 3 epochs), allowing adaptive optimization during training. The computational environment leveraged an NVIDIA A100 GPU via Google Colab Pro, facilitating accelerated training with a batch size of 64. Training was monitored using early stopping (patience = 5 epochs) to prevent overfitting while ensuring sufficient model convergence. This configuration balanced training efficiency with robust model development, providing a stable platform for systematic experimentation and performance evaluation. To not backbone network was frozen during testing then unfrozen for fine-tuning, and due to random weight initialisation, everything was ran more than once.

IIII. Model Selection

The study systematically evaluated multiple neural network architectures to identify the optimal model for CIFAR-100 classification. We began with a custom-designed CNN to establish a baseline.

A) Custom Neural Network

The custom convolutional neural network (CNN) architecture was designed as a progressive feature extractor with increasing channel depth and decreasing spatial resolution, following established CNN design principles [3]. The network begins with an input layer processing 32×32 RGB images, followed by three convolutional blocks 1) 32-channel 3×3 convolution (896 parameters) with 2×2 max pooling 2) 64-channel 3×3 convolution (18,496 parameters) with 2×2 max pooling 3) 128-channel 3×3 convolution (73,856 parameters) with 2×2 max pooling. Each convolutional layer employs ReLU activation and is immediately followed by max pooling with stride 2, systematically reducing spatial dimensions from 30×30 to 2×2 while expanding feature depth from 32 to 128 channels. The extracted features are flattened into a 512-element vector (2×2×128) and processed through 512-neuron fully connected layer (262,656 parameters) with ReLU activation and Dropout regularization (p=0.5) for improved generalization. [9] A 100-neuron output layer (51,300 parameters) with Softmax activation for class prediction. The architecture contains 406,204 trainable parameters total, with convolutional layers comprising 23% of parameters and dense layers 77%. This design follows the conventional pattern of early spatial feature extraction followed by high-level semantic processing [3], while maintaining computational efficiency suitable for baseline comparison with more complex architectures like ResNet [10].

B) ResNet50

The ResNet50V2 architecture was selected for this investigation due to its optimal balance between model capacity and computational efficiency within the ResNet family. As demonstrated in [10], while shallower variants (ResNet18/34) offer reduced computational complexity (3.6 GFLOPs vs. 7.8 GFLOPs for 224×224 inputs), their limited depth compromises feature extraction capability for complex 100-class classification tasks. Conversely, deeper architectures (ResNet101/152) demonstrate diminishing returns, with ResNet152 providing <2% accuracy improvement on ImageNet at 3x increased computational cost [10], [11].

It was selected as the backbone. So, it has 50 layers. For neurons, Conv1 has 64, Stage 1 has (256x3) 768, Stage 2 has 1536, Stage 3 has 6144, Stage 4 has 4608, and FC has 1000. Total of 25613800 parameters. For the classification head: for neurons, global average pooling reduces to 2048, dense has 512, and final dense (for classification) has 100; total parameters here is 1100388. In total, 16780 neurons and 26714188 parameters.

There are several strengths. Firstly, it has improved optimization and training stability, by using batch normalisation before activation, improving gradient flow, convergence, and effective deep network training. Secondly, it has good generalisation for small datasets, where 50 layers provide deep feature extraction and residual connections improve learning like hierarchical features and regularisation techniques can be applied for generalisation. Thirdly, it has balanced accuracy against computational costs. It achieves higher accuracy than shallower models and high accuracy with a lower computational footprint than larger models. Fourthly, it has strong transfer learning capabilities, as it was pre-trained on ImageNet, which can be fine-tuned on CIFAR-100 well with fewer training samples, without need for extensive training from scratch.

There are several weaknesses. Firstly, overhead for small images, as it was designed for larger 224x224 images, requiring CIFAR-100 images (32x32) to be upscaled which reduces efficiency ad may alter features. Secondly, it is computationally expensive, as is more demanding than ResNet18/34, needing more memory and FLOPs hence requires high-powered devices. Thirdly, it may have diminishing returns against shallower models, as they may achieve similar performance with less computation. Fourthly, it is not efficient, as requires significantly more resources than EfficientNet/MobileNet for only marginal accuracy gain, though it was relatively quick using an A100 GPU.

There are a few best uses cases. Firstly, for research and competitions, where test accuracy is prioritised over computational cost. Secondly, for pre-trained feature extraction, for fine-tuning ImageNet-trained models for CIFAR-100. Thirdly, cloud-based applications, where

computational cost is not a major constraint, where we used Google Colab.

However, there are some limitations. Firstly, it is prone to overlearning on CIFAR-100 due to the small size, hence regularisation or cross-validation required. Secondly, better alternatives exist like WideResNet which are more optimised for CIFAR-100. Thirdly, it may require modifications like architecture adjustments for small image sized or different number of classes.

IV. Model Performance

A) Custom Neural Network

Table 1 - Performance of custom CNN

Input	Accuracy	Loss	Precision	F1-Score
32x32	43.68%	2.2409	0.4356	0.4311
32x32 + Augmentation	43.60%	2.1363	0.4454	0.4300
224x224	33.29%	2.6920	0.3351	0.3242
224x224 + Augmentation	41.19%	2.3006	0.4126	0.4035

The custom neural network results in Table 1 demonstrated varying performance across different input configurations, with the native 32×32 resolution achieving the highest accuracy (43.68%) and lowest loss (2.2409). While data augmentation on 32×32 inputs slightly reduced accuracy to 43.60%, it improved precision (0.4454) and lowered loss (2.1363), suggesting better regularization. The upscaled 224×224 inputs showed significantly poorer performance (33.29% accuracy), indicating resolution mismatch challenges, though augmentation partially mitigated this effect (41.19% accuracy). Across all configurations, the F1-scores closely tracked accuracy values (0.3242-0.4311), confirming consistent class-wise performance without substantial prediction biases. The results suggest that maintaining native resolution with augmentation yields optimal balance between accuracy (43.60%) and model stability (lowest loss of 2.1363).

B) ResNet50V2 Initial and fine-tuning training

To effectively evaluate the models on CIFAR-100, a combination of metrics was used to address the multi-class classification nature of the problem.

Table 1 below highlights the decision of resizing our images to [224, 224], aligning with ResNet50V2's original input size:

Table 2 - Performance of ResNet50V2 with Different Input Sizes.

		Initial				Fine-tune			
Input size	Dropout	Accuracy	Loss	Precision	F1-score	Accuracy	Loss	Precision	F1-score
32x32	0.4	0.1658	3.634	0.5489	0.1508	0.1526	3.980	0.4346	0.1437
128x128	0.4	0.4103	2.241	0.6832	0.3992	0.3128	2.720	0.3729	0.2966
224x224	0.4	0.6334	1.312	0.7897	0.6314	0.6736	1.205	0.7813	0.6722

Table 2 below presents the results of the ResNet50V2 model at various stages of the study, showcasing the progression of our decision-making process and understanding of what worked better. The model was initially trained with a learning rate of 1e-3, followed by fine-tuning with a learning rate of 1e-5, and Adam optimiser is used throughout:

Table 3 - Performance of ResNet50V2 with Different Hyperparameters.

				Initial				Fine-tune			
Augment	Dropout	Weight	L2	Accuracy	Loss	Precision	F1-score	Accuracy	Loss	Precision	F1-score
---	---	---	---	0.6334	1.312	0.7897	0.6314	0.6736	1.210	0.7813	0.6722
Both same	0.4	---	---	0.6337	1.317	0.7819	0.6322	0.7824	0.803	0.8296	0.7816
Both diff	0.8	0.9	0.9	0.5769	1.559	0.8585	0.5692	0.6290	1.323	0.8569	0.6266
Fine-tune	0.6	0.3	0.9	0.6322	1.276	0.8061	0.6293	0.6704	1.145	0.8991	0.6689
Both diff	0.7	0.03	0.9	0.6431	1.289	0.8032	0.6394	0.6628	1.241	0.7990	0.6621
Fine-Tune	0.7	* 0.3	0.9	0.6272	1.307	0.8284	0.6231	0.6597	1.199	0.8375	0.6580

*fine-tune as well

Note that for all results in Table 1, weight decay was applied during the initial training, and L2 regularisation was implemented across all layers.

While a learning rate of 1e-3 is typically used for initial training, we also explored different values for the ResNet50V2 model, as shown in Table 3:

Table 4 - Performance of ResNet50V2 with Different Learning Rates and Hyperparameters.

					Initial				Fine-tune			
Augment	Dropout	Weight	L2	LR	Accuracy	Loss	Precision	F1-score	Accuracy	Loss	Precision	F1-score
---	0.4	---	---	3e-3 1e-5	0.5748	1.586	0.7607	0.5730	0.6479	1.374	0.7680	0.6474
---	---	---	---	3e-3 1e-5	0.5880	5.029	0.6479	0.5873	0.5883	5.110	0.6454	0.5927
fine-tune	0.7	0.3	0.3	5e-3 1e-5	0.4448	2.054	0.7945	0.4233	0.5443	1.709	0.8564	0.5397
fine-tune	0.7	0.3	0.3	5e-3 1e-4	0.4376	2.085	0.7877	0.4057	0.6213	1.378	0.8436	0.6188

Table 4 below demonstrates several runs using augmentation for both initial and fine-tuning training, showing its benefit:

Table 5 - Performance of ResNet50V2 with Augmentation and Different Hyperparameters.

			Initial				Fine-tune			
Dropout	Weight	L2	Accuracy	Loss	Precision	F1-score	Accuracy	Loss	Precision	F1-score
0.6	0.03	---	0.6388	1.298	0.7891	0.5378	0.6657	1.220	0.7964	0.6648
0.7	0.03	0.3	0.6431	1.289	0.8032	0.6394	0.6628	1.241	0.7990	0.6621
0.6	0.03	---	0.6361	1.296	0.7885	0.6375	0.6603	1.179	0.8041	0.6598
0.8	0.90	0.9	0.5769	1.559	0.8585	0.5692	0.6290	1.323	0.8569	0.6266

Based on the accuracy and other metrics in the tables, ResNet50V2 achieved the highest testing accuracy of 67.04%, with dropout set at 0.6, weight decay at 0.3, and L2 regularisation at 0.9, though it did over-regularise during fine-tuning.

The higher fine-tuning accuracy reaching above 67.04% can be attributed to overfitting during initial training. Below are the plots of ResNet50V2 models overfitting and the best model:

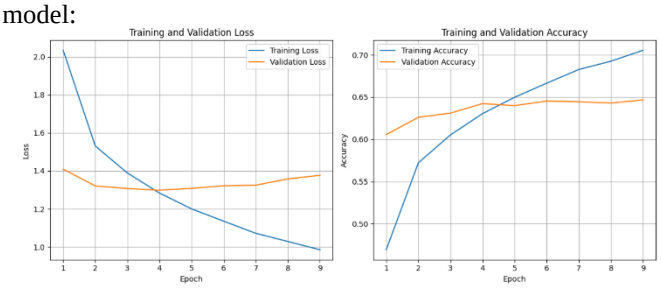

Figure 3 - ResNet50V2 Model Overfitting During Initial Training.

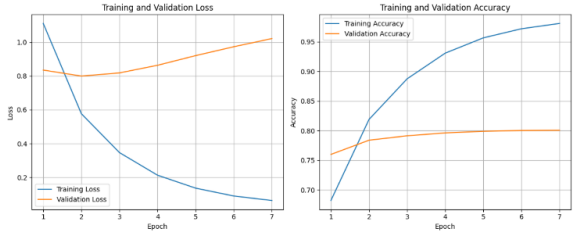

Figure 4 - ResNet50V2 Model Overfitting During Fine-Tuning.

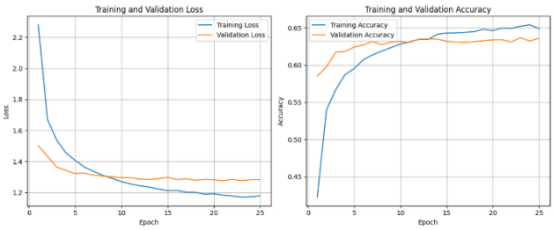

Figure 5 - Best ResNet50V2 Model Performance During Initial Training.

Figure 6 - Best ResNet50V2 Model Performance During Fine-Training.

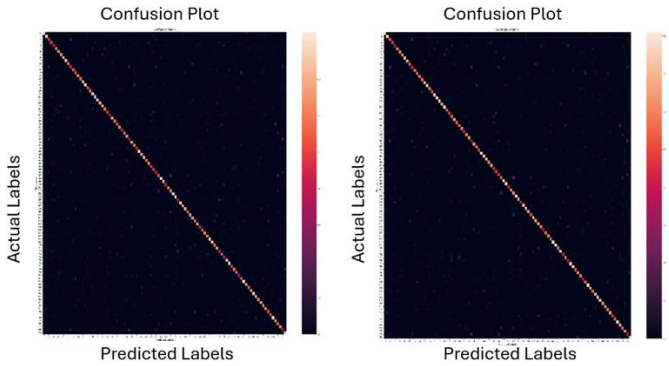

Figure 7 - Confusion Matrices for Overfitting and Best Models.

The confusion matrices for Figures 4 and 6 are shown below, however both models show similar class-wise performance, with a general spread of misclassifications rather than specific classes being misidentified.

LeNet achieved an accuracy of just under 20%. In comparison, the ResNet50V2 models demonstrated significantly higher accuracy, highlighting that deeper models tend to perform better as they can learn more detailed and intricate features. When VGG16 was trained from scratch, it only achieved around 1% accuracy after a significant duration relative to ResNet50V2, further emphasising the power of transfer learning.

Table 1 illustrates that resizing images to match the input dimensions expected by the model improves accuracy, due to correctly sized images allow the model to extract features more effectively.

Table 2 presents insights into the learning process and impact of overfitting. Rows 1 and 2 yielded higher accuracy than the best-performing model in row 4, but this was due to overfitting, meaning the models failed to generalise well to the test dataset. The overfitting of rows 1 and 2 is evident in Figures 4 and 5, where the loss curve rises, and the training accuracy significantly exceeds the validation accuracy. In contrast, row 4 shows a more stable learning process, as seen in Figure 6, where the loss remains stable, and the training and validation accuracy are closely aligned. However, Figure 7 suggests over-regularisation, where the validation accuracy is much lower than the training accuracy, indicating that while the model initially appeared well-generalised, excessive regularisation may have hindered its learning ability.

As expected, increasing dropout led to improved accuracy, as it forces the remaining neurons to learn more independent and diverse representations, hence mitigating overfitting. A similar effect was observed with data augmentation, which enhanced model generalisation and robustness by increasing the diversity and volume of training data. A larger initial learning rate was detrimental due to being too large to locate smaller improvements, whilst a larger fine-tuning learning rate increased accuracy, but due to overfitting.

To address initial overfitting, adjustments were made to explore the model's limitations and identify factors contributing to underfitting. It was found that excessive weight decay (0.9) and dropout (0.9) negatively impacted performance by preventing effective learning. However, by finding an optimal balance, a dropout rate of 0.6 and weight decay of 0.3 was determined to be ideal. These settings resulted in higher and more stable accuracy, with training and validation closely aligned, indicating better generalisation.

F1 scores were generally very similar to accuracy, due to the datasets being well balanced. Overall, the models did successively achieve over 60%, with the models with best balance providing good generalisation and robustness to the datasets.

V. DICUSSION

A) Key Findings

The study highlights the impact of image resizing, dropout, weight decay, learning rate and data augmentation in enhancing model accuracy and generalisation. It also emphasizes that deeper models like ResNet50V2 outperform simpler architectures like LeNet particularly when trained using transfer learning. The ability of the models to achieve over 60% accuracy with the best-balanced configurations demonstrating strong generalisation, suggests practical applicability in real-world classification problems. The implications of these findings are twofold: model selection matters, as there is a significant gap between shallower and deeper networks for complex images classification; and generalisation vs overfitting trade-off, as excessive regularisation can hinder learning, but optimised settings improve both accuracy and robustness. Robustness is models' ability to be accurate across diverse dataset and avoid overfitting. The confusion matrices analysis indicates that classifications were generally spread out rather than concentrated, suggesting an unbiased model, however overall classification robustness still requires improvements.

B) Performance Analysis

Achieving 60% accuracy indicates a reasonable level of performance though room for improvement. The study shows that models with optimised dropout (0.6) and weight decay (0.3) achieved the best generalisation, reinforcing the idea that hyperparameter tuning is crucial

for balancing performance. Moreover, the F1 scores closely mirroring accuracy confirms well balanced class distributions, reducing concerns about skewed predictions. The study's findings also align with well-established DL best practises, where data augmentation and dropout enhance generalisation, while excessive regularisation hinders learning.

C) *Methodological Strengths*

The study's strengths are three-fold. Firstly, hyperparameter optimisation, where the impact of dropout, weight decay and regularisation were rigorously analysed, with other hyperparameters also being explored, providing practical guidelines for improving classification models. Secondly, the use of transfer learning, with its power showing through the performance increase compared to models from scratch. Thirdly, generalisation and robustness analysis, where the study effectively discussed overfitting, underfitting, and the balance between them for how to achieve stable model performance.

D) *Limitations*

There are several limitations. Firstly, the accuracy remains moderate, where further improvements could be found through fine-tuning or more sophisticated or deeper models. Secondly, lack of extensive dataset variability, as it is unclear if the model would generalise well to highly imbalanced real-world datasets. Thirdly, a more comprehensive confusion matrix analysis could provide deeper understanding on potential biases within the data.

E) *Recommendations*

There are several recommendations. Firstly, exploring additional architectures, like vision transformers, or fine-tune hyperparameters to increase accuracy. Secondly, combining models may enhance robustness and reduce generalisation errors. Different optimiser could be tested, such as AdamW which is an improved version of Adam.

VI. Conclusion

This study effectively demonstrated the advantages of deeper architectures, transfer learning, and hyperparameter tuning in image classification tasks. Deeper models provide far superior accuracy and confidence due to ability to learn more intricate features. There is also a delicate balance of hyperparameters to prevent overfitting whilst preventing over-regularisation. Overall, our best model provided decent satisfactory accuracy with good robustness, whilst exploring how models can be configured, and strategies can be altered to achieve optimised results.

VII. References

Both students explored, developed and optimized different runs, models and hyperparameters. Both students also wrote the report.

Andrew Law: 50%

Pittayut Benjamasutin: 50%

References

[1] Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in Advances in Neural Information Processing Systems, vol. 25, 2012, pp. 1097–1105. [Online]. Available: https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

[2]"CIFAR-10 and CIFAR-100 datasets," [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html, accessed: Mar. 20, 2025.

[3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.

[4] S. Ruder, "An overview of gradient descent optimization algorithms," arXiv:1609.04747, Jun. 2017. [Online]. Available: https://arxiv.org/abs/1609.04747

[5] G. Hinton, "Neural networks for machine learning," [Lecture slides]. [Online]. Available: https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, Jan. 2017. [Online]. Available: https://arxiv.org/abs/1412.6980

[7] "Papers with Code - CIFAR-100 benchmark (image classification)," [Online]. Available: https://paperswithcode.com/sota/image-classification-on-cifar-100, accessed: Mar. 20, 2025.

[8] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," arXiv:1502.03167, Mar. 2015. [Online]. Available: https://arxiv.org/abs/1502.03167

[9] A. Y. Ng, "Feature selection, L1 vs. L2 regularization, and rotational invariance," in Proc. 21st Int. Conf. Mach. Learn. (ICML), 2004, p. 78, doi: 10.1145/1015330.1015435.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," arXiv:1512.03385, Dec. 2015. [Online]. Available: https://arxiv.org/abs/1512.03385

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," arXiv:1603.05027, Jul. 2016. [Online]. Available: https://arxiv.org/abs/1603.05027

[12] Al-Humaidan, Norah & Prince, Master. (2021). A Classification of Arab Ethnicity Based on Face Image Using Deep Learning Approach. IEEE Access. PP. 1-1. 10.1109/ACCESS.2021.3069022.

APPENDIX A

Github repo of all runs.
https://github.com/andrewlaw9178/EE992_CIFAR-100

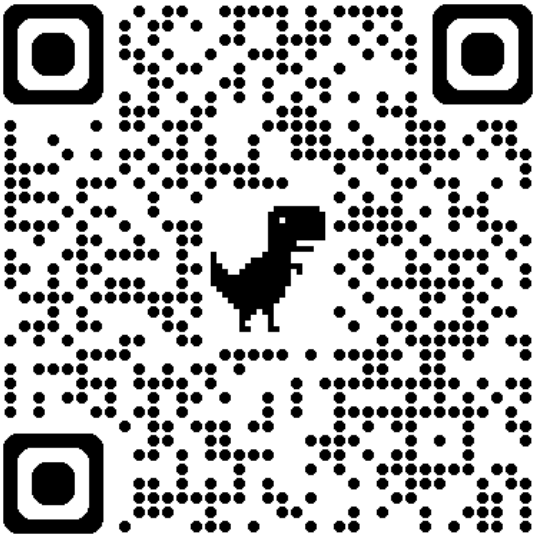

Table 2:
- All from ResNet50_Update_Test

Table 3:
- Row 1 from ResNet50_Update_Test
- Row 2 from ResNet50_Update2_Test
- Row 3 from ResNet50_Update4_Test
- Row 4 from ResNet50_Update6_Test
- Row 5 from ResNet50_Update3_Test
- Row 6 from ResNet50_Update3_5_Test

Table 4:
- Row 1 from ResNet50_Update4_Test
- Row 2 from ResNet50_Update6_Test
- Row 3 from ResNet50_Update6_Test
- Row 4 from ResNet50_Update5_Test

Table 5:
- Row 1 from ResNet50_Update2_Test
- Row 2 from ResNet50_Update3_Test
- Row 3 from ResNet50_Update3_5_Test
- Row 4 from ResNet50_Update4_Test